Multi-Armed Bandits

Qinzhen Li, Jussi Keppo, Hong Ming Tan, and Linsheng Zhuang

National University of Singapore

2025

Learning to Act Under Uncertainty

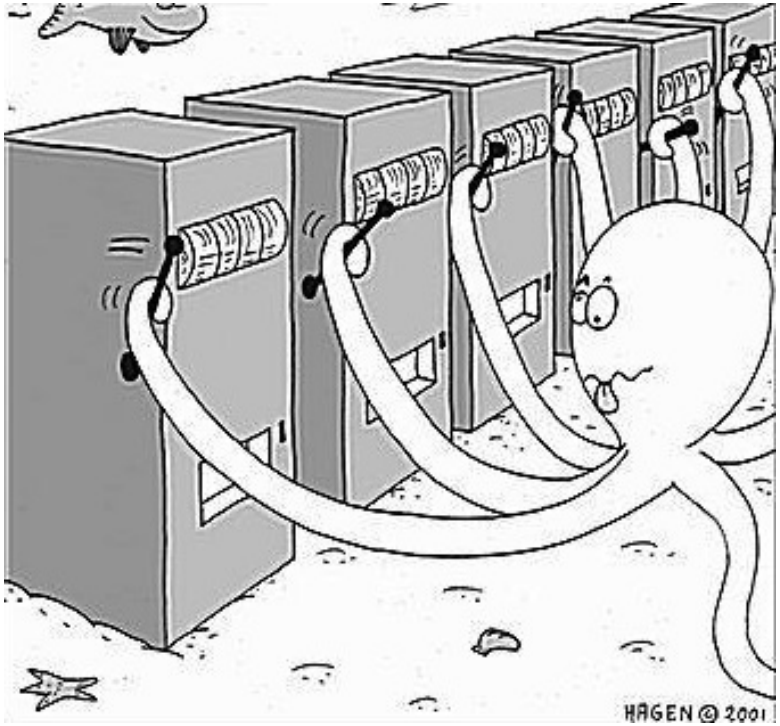
- You're Netflix: which movie should you recommend next?

Learning to Act Under Uncertainty

- You're Netflix: which movie should you recommend next?
- You're an online store: what price should you offer a new customer?

Learning to Act Under Uncertainty

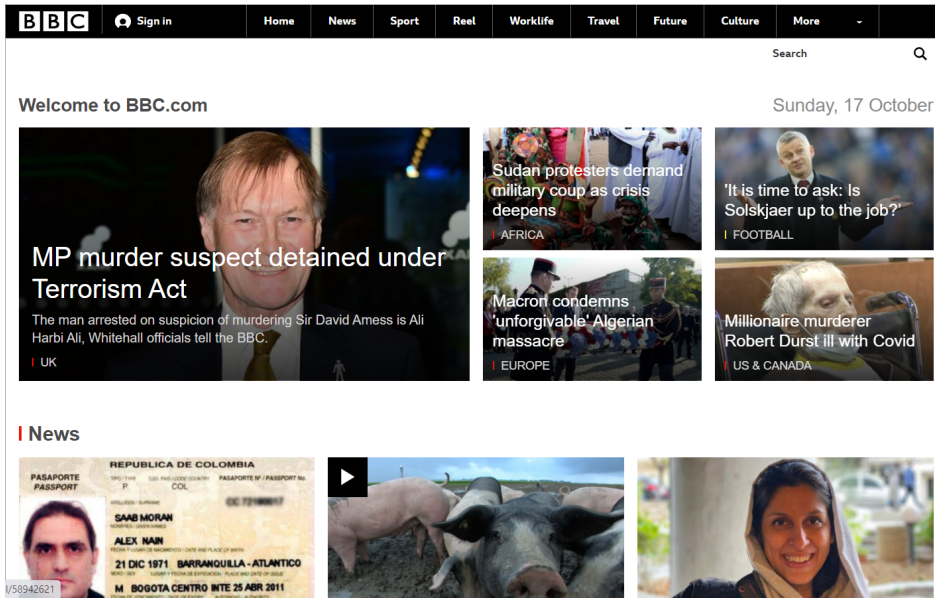
- You're Netflix: which movie should you recommend next?
- You're an online store: what price should you offer a new customer?
- You're a doctor: which treatment should you try for a new patient?



- You are standing in front of three slot machines. Each machine gives a reward with its own (unknown) probability. Some pay out more often than others.
- Your goal: figure out which arm is the best one to pull.
- A game: `https://cse442-17f.github.io/LinUCB/`

Examples

concrete albeit very stylized



News site When a new visitor arrives, the website selects article to display and observes whether the visitor clicks on it.

Examples

concrete albeit very stylized

Pricing When a customer enters, the store offers a price and observes whether the customer purchases or not.

Examples

concrete albeit very stylized

Pricing When a customer enters, the store offers a price and observes whether the customer purchases or not.

Investment Each morning, you choose one stock to invest. At the end of the day, you observe how all the stocks performed.

Canteen ?

The basic idea

- K possible actions/arms
- T rounds
- Each round, you choose an arm and collect a reward for this arm

The basic idea

- K possible actions/arms
- T rounds
- Each round, you choose an arm and collect a reward for this arm
- For each arm, reward is drawn **independently** from an unknown distribution which is **fixed** across time (i.i.d rewards, depends only on the chosen arm)

Back to our examples

Example	**Action**	**Reward**
News site	Choose an article to display	1 if clicked, 0 otherwise
Pricing	Offer a price p	p if purchased, 0 otherwise
Investment	Choose a stock to invest in	Change in value during the day
Canteen	?	?

Exploration vs. Missing Information

- After pulling an arm, you observe its reward.
- But you learn nothing about the rewards from arms you didn't choose. So, what should we do?

Exploration vs. Missing Information

- After pulling an arm, you observe its reward.
- But you learn nothing about the rewards from arms you didn't choose. So, what should we do?
- The algorithm must *explore*: try different arms to gather information.
- After all, if you always eat at the same food stall, how would you know if another one tastes better?

Exploration vs. Exploitation

- In multi-armed bandit (MAB) problems, we face a fundamental trade-off:
  - **Exploration** (gathering information);
  - **Exploitation** (using what we know to make the best decision).

Exploration vs. Exploitation

- In multi-armed bandit (MAB) problems, we face a fundamental trade-off:
  - **Exploration** (gathering information);
  - **Exploitation** (using what we know to make the best decision).
- This balance is key:
  - explore too little, and you might miss better options;
  - explore too much, and you miss out on rewards.

Exploration vs. Exploitation

- In multi-armed bandit (MAB) problems, we face a fundamental trade-off:
  - **Exploration** (gathering information);
  - **Exploitation** (using what we know to make the best decision).
- This balance is key:
  - explore too little, and you might miss better options;
  - explore too much, and you miss out on rewards.
- A good algorithm learns which arms perform best, while limiting unnecessary exploration.

Auxiliary Feedback

Sometimes, we get more information for free

- What additional feedback (if any) is available after each round, beyond the reward from the chosen arm?

Auxiliary Feedback

Sometimes, we get more information for free

- What additional feedback (if any) is available after each round, beyond the reward from the chosen arm?

Example	Auxiliary Feedback	Rewards for Other Arms?
News site	None	No (bandit feedback)

Auxiliary Feedback

Sometimes, we get more information for free

- What additional feedback (if any) is available after each round, beyond the reward from the chosen arm?

Example	**Auxiliary Feedback**	**Rewards for Other Arms?**
News site	None	No (bandit feedback)
Pricing	If sale: customer would also buy at a lower price. If no sale: customer would not buy at any higher price.	Yes, for some arms (partial feedback)

Auxiliary Feedback

Sometimes, we get more information for free

- What additional feedback (if any) is available after each round, beyond the reward from the chosen arm?

Example	**Auxiliary Feedback**	**Rewards for Other Arms?**
News site	None	No (bandit feedback)
Pricing	If sale: customer would also buy at a lower price. If no sale: customer would not buy at any higher price.	Yes, for some arms (partial feedback)
Investment	You observe the change in value for all stocks.	Yes, for all arms (full feedback)

Auxiliary Feedback

Sometimes, we get more information for free

- What additional feedback (if any) is available after each round, beyond the reward from the chosen arm?

Example	**Auxiliary Feedback**	**Rewards for Other Arms?**
News site	None	No (bandit feedback)
Pricing	If sale: customer would also buy at a lower price. If no sale: customer would not buy at any higher price.	Yes, for some arms (partial feedback)
Investment	You observe the change in value for all stocks.	Yes, for all arms (full feedback)
Canteen	?	?

Three Types of Feedback

Bandit feedback	**Only the reward of the chosen arm** is observed.
Partial feedback	The algorithm observes the reward of the chosen arm **plus limited information** about other arms.
Full feedback	**Rewards for all arms** are observed, regardless of which one is chosen.

Three Types of Feedback

Bandit feedback	**Only the reward of the chosen arm** is observed.
Partial feedback	The algorithm observes the reward of the chosen arm **plus limited information** about other arms.
Full feedback	**Rewards for all arms** are observed, regardless of which one is chosen.

We will focus on problems with **bandit feedback**.

Contextual Bandits

- In each round, we observe *context* before choosing an action.
  - Often includes known information about the user or environment.
  - **Personalized decisions**: what works best depend on the context.

Contextual Bandits

- In each round, we observe *context* before choosing an action.
  - Often includes known information about the user or environment.
  - **Personalized decisions**: what works best depend on the context.
- **Examples of context in different applications:**

Example	Context
News site	User location, device type, and browsing history
Pricing	Store location, time of day, and customer demographics
Investment	Current economic indicators or market trends
Canteen	?

Contextual Bandits

- In each round, we observe *context* before choosing an action.
  - Often includes known information about the user or environment.
  - **Personalized decisions**: what works best depend on the context.
- **Examples of context in different applications:**

Example	Context
News site	User location, device type, and browsing history
Pricing	Store location, time of day, and customer demographics
Investment	Current economic indicators or market trends
Canteen	?

- Instead of learning the best arm, we learn the best **policy**, a rule for choosing the best action **given the context**.

From Intuition to Algorithms

Reward Models

Where do the rewards come from?

I.I.D.

Each arm's reward is drawn **independently** from a fixed distribution. The distribution **does not change** over time.

Reward Models

Where do the rewards come from?

I.I.D.	Each arm's reward is drawn **independently** from a fixed distribution. The distribution **does not change** over time.
Adversarial	Rewards can be **arbitrarily**, as if by an "adversary" trying to fool the algorithm.

Reward Models

Where do the rewards come from?

I.I.D.	Each arm's reward is drawn **independently** from a fixed distribution. The distribution **does not change** over time.
Adversarial	Rewards can be **arbitrarily**, as if by an "adversary" trying to fool the algorithm.
Constrained adversary	Adversarial rewards, but with **limits**, e.g., each arm's reward can change only a few times.

Reward Models

Where do the rewards come from?

I.I.D.	Each arm's reward is drawn **independently** from a fixed distribution. The distribution **does not change** over time.
Adversarial	Rewards can be **arbitrarily**, as if by an "adversary" trying to fool the algorithm.
Constrained adversary	Adversarial rewards, but with **limits**, e.g., each arm's reward can change only a few times.
Stochastic (non-I.I.D.)	Rewards evolve over time as a stochastic process, e.g., a random walk.

Stochastic Bandits and Notation

- Rewards are **i.i.d.** over time.

Stochastic Bandits and Notation

- Rewards are **i.i.d.** over time.
- There are K possible actions (arms) denoted by $a \in \mathcal{A}$.

Stochastic Bandits and Notation

- Rewards are **i.i.d.** over time.
- There are K possible actions (arms) denoted by $a \in \mathcal{A}$.
- The interaction lasts for T rounds, indexed by $t \in [T]$.

Stochastic Bandits and Notation

- Rewards are **i.i.d.** over time.
- There are K possible actions (arms) denoted by $a \in \mathcal{A}$.
- The interaction lasts for T rounds, indexed by $t \in [T]$.
- Pulling arm a at time t yields a reward $r_t^a \sim \mathcal{D}_a$, where $\mathcal{D}_a$ is an *unknown* distribution.

Stochastic Bandits and Notation

- Rewards are **i.i.d.** over time.
- There are K possible actions (arms) denoted by $a \in \mathcal{A}$.
- The interaction lasts for T rounds, indexed by $t \in [T]$.
- Pulling arm a at time t yields a reward $r_t^a \sim \mathcal{D}_a$, where $\mathcal{D}_a$ is an *unknown* distribution.
- The mean reward of arm a is $\mu(a) := \mathbb{E}_{r_t^a \sim \mathcal{D}_a}[r_t^a]$.

Stochastic Bandits and Notation

- Rewards are **i.i.d.** over time.
- There are K possible actions (arms) denoted by $a \in \mathcal{A}$.
- The interaction lasts for T rounds, indexed by $t \in [T]$.
- Pulling arm a at time t yields a reward $r_t^a \sim \mathcal{D}_a$, where $\mathcal{D}_a$ is an *unknown* distribution.
- The mean reward of arm a is $\mu(a) := \mathbb{E}_{r_t^a \sim \mathcal{D}_a}[r_t^a]$.
- The best achievable mean reward is

$$\mu^* := \max_{a \in \mathcal{A}} \mu(a), \quad a^* = \arg\max_{a \in \mathcal{A}} \mu(a).$$

Problem Protocol: Multi-Armed Bandits

Setup

- K arms (actions), T rounds.
- In each round $t \in [T]$:
  1. The algorithm selects an arm $a_t \in \mathcal{A}$.
  2. It observes a reward $r_t \in [0, 1]$ for the chosen arm.

Problem Protocol: Multi-Armed Bandits

Setup

- K arms (actions), T rounds.
- In each round $t \in [T]$:
  1. The algorithm selects an arm $a_t \in \mathcal{A}$.
  2. It observes a reward $r_t \in [0, 1]$ for the chosen arm.
- The goal is to maximize the **total reward** $\sum_{t=1}^{T} r_t$.

Problem Protocol: Multi-Armed Bandits

Setup

- K arms (actions), T rounds.
- In each round $t \in [T]$:
  1. The algorithm selects an arm $a_t \in \mathcal{A}$.
  2. It observes a reward $r_t \in [0, 1]$ for the chosen arm.
- The goal is to maximize the **total reward** $\sum_{t=1}^{T} r_t$.

Assumptions

- Only the reward for the chosen arm is observed (*bandit feedback*).

Problem Protocol: Multi-Armed Bandits

Setup

- K arms (actions), T rounds.
- In each round $t \in [T]$:
  1. The algorithm selects an arm $a_t \in \mathcal{A}$.
  2. It observes a reward $r_t \in [0, 1]$ for the chosen arm.
- The goal is to maximize the **total reward** $\sum_{t=1}^{T} r_t$.

Assumptions

- Only the reward for the chosen arm is observed (*bandit feedback*).
- Rewards for each arm are **i.i.d.** over time.

Problem Protocol: Multi-Armed Bandits

Setup

- K arms (actions), T rounds.
- In each round $t \in [T]$:
  1. The algorithm selects an arm $a_t \in \mathcal{A}$.
  2. It observes a reward $r_t \in [0, 1]$ for the chosen arm.
- The goal is to maximize the **total reward** $\sum_{t=1}^{T} r_t$.

Assumptions

- Only the reward for the chosen arm is observed (*bandit feedback*).
- Rewards for each arm are **i.i.d.** over time.
- Per-round rewards are bounded, typically $r_t \in [0, 1]$.

How to know if an algorithm is doing a good job?

Regret

$$R(T) = \mu^* \cdot T - \sum_{t=1}^{T} \mu(a_t)$$

- This quantity is called **regret** at round T.

Regret

$$R(T) = \mu^* \cdot T - \sum_{t=1}^{T} \mu(a_t)$$

- This quantity is called **regret** at round T.
- a_t is random, as it depends on randomness in rewards and/or in the algorithm. Hence $R(T)$ is also random.
- We typically consider the **expected regret:** $\mathbb{E}[R(T)]$.

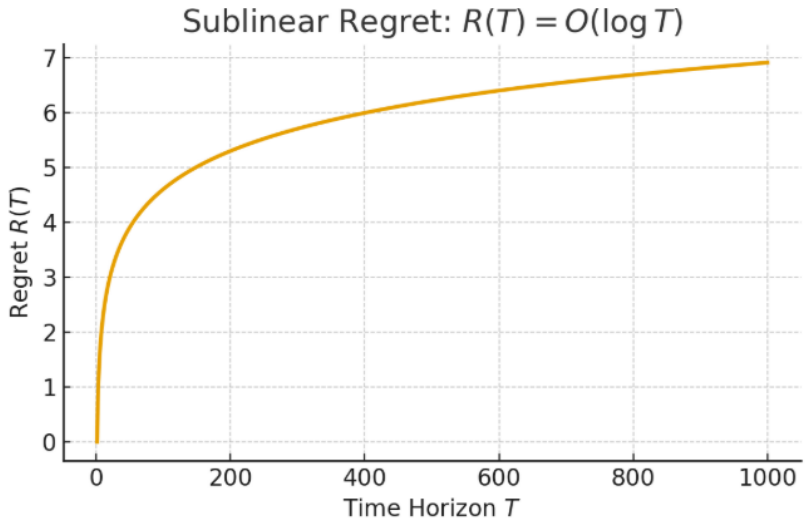
Regret

$$R(T) = \mu^* \cdot T - \sum_{t=1}^{T} \mu(a_t)$$

- This quantity is called **regret** at round T.
- a_t is random, as it depends on randomness in rewards and/or in the algorithm. Hence $R(T)$ is also random.
- We typically consider the **expected regret:** $\mathbb{E}[R(T)]$.
- The goal is to design algorithms where $\mathbb{E}[R(T)]$ grows *sublinearly* in T.

Regret

$$R(T) = \mu^* \cdot T - \sum_{t=1}^{T} \mu(a_t)$$



- This quantity is called **regret** at round T.
- a_t is random, as it depends on randomness in rewards and/or in the algorithm. Hence $R(T)$ is also random.
- We typically consider the **expected regret:** $\mathbb{E}[R(T)]$.
- The goal is to design algorithms where $\mathbb{E}[R(T)]$ grows *sublinearly* in T.

What is the simplest algorithm you can think of?

What is the simplest algorithm you can think of?

Try everything equally.

Uniform Exploration

Algorithm 1.1: Explore-First with parameter N

1. **Exploration phase:** try each arm N times.
2. **Selection:** pick the arm $\hat{a}$ with the highest empirical mean reward.
3. **Exploitation phase:** play $\hat{a}$ for all remaining rounds.

Uniform Exploration

Algorithm 1.1: Explore-First with parameter N

1. **Exploration phase:** try each arm N times.
2. **Selection:** pick the arm $\hat{a}$ with the highest empirical mean reward.
3. **Exploitation phase:** play $\hat{a}$ for all remaining rounds.

- Fixed period of exploration.

Uniform Exploration

Algorithm 1.1: Explore-First with parameter N

1. **Exploration phase:** try each arm N times.
2. **Selection:** pick the arm $\hat{a}$ with the highest empirical mean reward.
3. **Exploitation phase:** play $\hat{a}$ for all remaining rounds.

- Fixed period of exploration.
- **What could go wrong?**

Uniform Exploration

Algorithm 1.1: Explore-First with parameter N

1. **Exploration phase:** try each arm N times.
2. **Selection:** pick the arm $\hat{a}$ with the highest empirical mean reward.
3. **Exploitation phase:** play $\hat{a}$ for all remaining rounds.

- Fixed period of exploration.
- **What could go wrong?**

  If N is too small $\Rightarrow$ not enough learning;

  if N is too large $\Rightarrow$ too much exploration.

Uniform Exploration

Algorithm 1.1: Explore-First with parameter N

1. **Exploration phase:** try each arm N times.
2. **Selection:** pick the arm $\hat{a}$ with the highest empirical mean reward.
3. **Exploitation phase:** play $\hat{a}$ for all remaining rounds.

- Fixed period of exploration.
- **What could go wrong?**

  If N is too small $\Rightarrow$ not enough learning;

  if N is too large $\Rightarrow$ too much exploration.
- How could we improve this algorithm?

ε-Greedy Algorithm

Algorithm

At each round t:

- With probability ε, choose an arm uniformly at random (**explore**).

ε-Greedy Algorithm

Algorithm

At each round t:

- With probability ε, choose an arm uniformly at random (**explore**).
- With probability $1 - \varepsilon$, choose the arm with the highest average reward so far (**exploit**).

ε-Greedy Algorithm

Algorithm

At each round t:

- With probability ε, choose an arm uniformly at random (**explore**).
- With probability $1 - \varepsilon$, choose the arm with the highest average reward so far (**exploit**).

- Simple and effective when ε is well-chosen.

ε-Greedy Algorithm

Algorithm

At each round t:

- With probability ε, choose an arm uniformly at random (**explore**).
- With probability $1 - \varepsilon$, choose the arm with the highest average reward so far (**exploit**).

- Simple and effective when ε is well-chosen.
- **But:** fixed ε means exploration never stops, even when we already know a good arm.

ε-Greedy Algorithm

Algorithm

At each round t:

- With probability ε, choose an arm uniformly at random (**explore**).
- With probability $1 - \varepsilon$, choose the arm with the highest average reward so far (**exploit**).

- Simple and effective when ε is well-chosen.
- **But:** fixed ε means exploration never stops, even when we already know a good arm.
- Can we make ε *adaptive* as we learn?

Adaptive ε-Greedy

Algorithm

Use a **decaying** exploration rate ε_t:

- With probability ε_t, explore (choose an arm uniformly at random).
- With probability $1 - \varepsilon_t$, exploit (choose the best arm so far).

Adaptive ε-Greedy

Algorithm

Use a **decaying** exploration rate ε_t:

- With probability ε_t, explore (choose an arm uniformly at random).
- With probability $1 - \varepsilon_t$, exploit (choose the best arm so far).

Theory

For a suitable choice of $\varepsilon_t = \frac{1}{t}$, the expected regret satisfies

$$\mathbb{E}[R(T)] = O(\log T),$$

as shown in Auer et al. (2002) and Cesa-Bianchi and Fischer (1998).

Empirical Finding: Simplicity Wins

Vermorel and Mohri (2005) **found no practical advantage** in using adaptive ε-greedy variants.

> *"The most naive approach, the ε-greedy strategy, proves to be often hard to beat."*

From ε-Greedy to UCB

- ε-greedy explores **at random**: it doesn't consider which arms are uncertain.

From ε-Greedy to UCB

- ε-greedy explores **at random**: it doesn't consider which arms are uncertain.
- Some arms are well understood, others remain uncertain.

From ε-Greedy to UCB

- ε-greedy explores **at random**: it doesn't consider which arms are uncertain.
- Some arms are well understood, others remain uncertain.
- **Idea:** Explore when uncertain, exploit when confident.

From ε-Greedy to UCB

- ε-greedy explores **at random**: it doesn't consider which arms are uncertain.
- Some arms are well understood, others remain uncertain.
- **Idea:** Explore when uncertain, exploit when confident.
- Instead of random exploration, explore guided by uncertainty.

Upper Confidence Bound (UCB1)

Notation

- $n_a(t)$: number of times arm a has been chosen up to round t.
- $\hat{\mu}_a(t)$: empirical mean reward of arm a after $n_a(t)$ plays.

Upper Confidence Bound (UCB1)

Notation

- $n_a(t)$: number of times arm a has been chosen up to round t.
- $\hat{\mu}_a(t)$: empirical mean reward of arm a after $n_a(t)$ plays.

Idea

At each round, choose the arm that maximizes an **optimistic estimate** of its reward:

$$\text{UCB}_a(t) = \hat{\mu}_a(t) + \sqrt{\frac{2 \ln t}{n_a(t)}}.$$

The second term acts as an *exploration bonus* that shrinks as $n_a(t)$ grows.

Optimism in the face of uncertainty.

Upper Confidence Bound (UCB1)

Algorithm

1. **Initialization:** Play each arm once to collect initial rewards.

Upper Confidence Bound (UCB1)

Algorithm

1. **Initialization:** Play each arm once to collect initial rewards.
2. For each round $t = K + 1, \ldots, T$:

Upper Confidence Bound (UCB1)

Algorithm

1. **Initialization:** Play each arm once to collect initial rewards.
2. For each round $t = K + 1, \ldots, T$:
    1. Compute $\text{UCB}_a(t)$ for all arms a.

Upper Confidence Bound (UCB1)

Algorithm

1. **Initialization:** Play each arm once to collect initial rewards.
2. For each round $t = K + 1, \ldots, T$:
   1. Compute $\text{UCB}_a(t)$ for all arms a.
   2. Select the arm $\hat{a}_t = \arg\max_a \text{UCB}_a(t)$.

Upper Confidence Bound (UCB1)

Algorithm

1. **Initialization:** Play each arm once to collect initial rewards.
2. For each round $t = K + 1, \ldots, T$:
   1. Compute $\mathrm{UCB}_a(t)$ for all arms a.
   2. Select the arm $\hat{a}_t = \arg\max_a \mathrm{UCB}_a(t)$.
   3. Observe reward $r_t^{\hat{a}_t}$ and update:

$$\hat{\mu}_{\hat{a}_t}(t+1) = \frac{\hat{\mu}_{\hat{a}_t}(t)\, n_{\hat{a}_t}(t) + r_t^{\hat{a}_t}}{n_{\hat{a}_t}(t) + 1} \quad \text{and} \quad n_{\hat{a}_t}(t+1) = n_{\hat{a}_t}(t) + 1.$$

Upper Confidence Bound (UCB1)

Algorithm

1. **Initialization:** Play each arm once to collect initial rewards.
2. For each round $t = K + 1, \ldots, T$:
   1. Compute $\mathrm{UCB}_a(t)$ for all arms a.
   2. Select the arm $\hat{a}_t = \arg\max_a \mathrm{UCB}_a(t)$.
   3. Observe reward $r_t^{\hat{a}_t}$ and update:

$$\hat{\mu}_{\hat{a}_t}(t+1) = \frac{\hat{\mu}_{\hat{a}_t}(t)\, n_{\hat{a}_t}(t) + r_t^{\hat{a}_t}}{n_{\hat{a}_t}(t) + 1} \quad \text{and} \quad n_{\hat{a}_t}(t+1) = n_{\hat{a}_t}(t) + 1.$$

Upper Confidence Bound (UCB1)

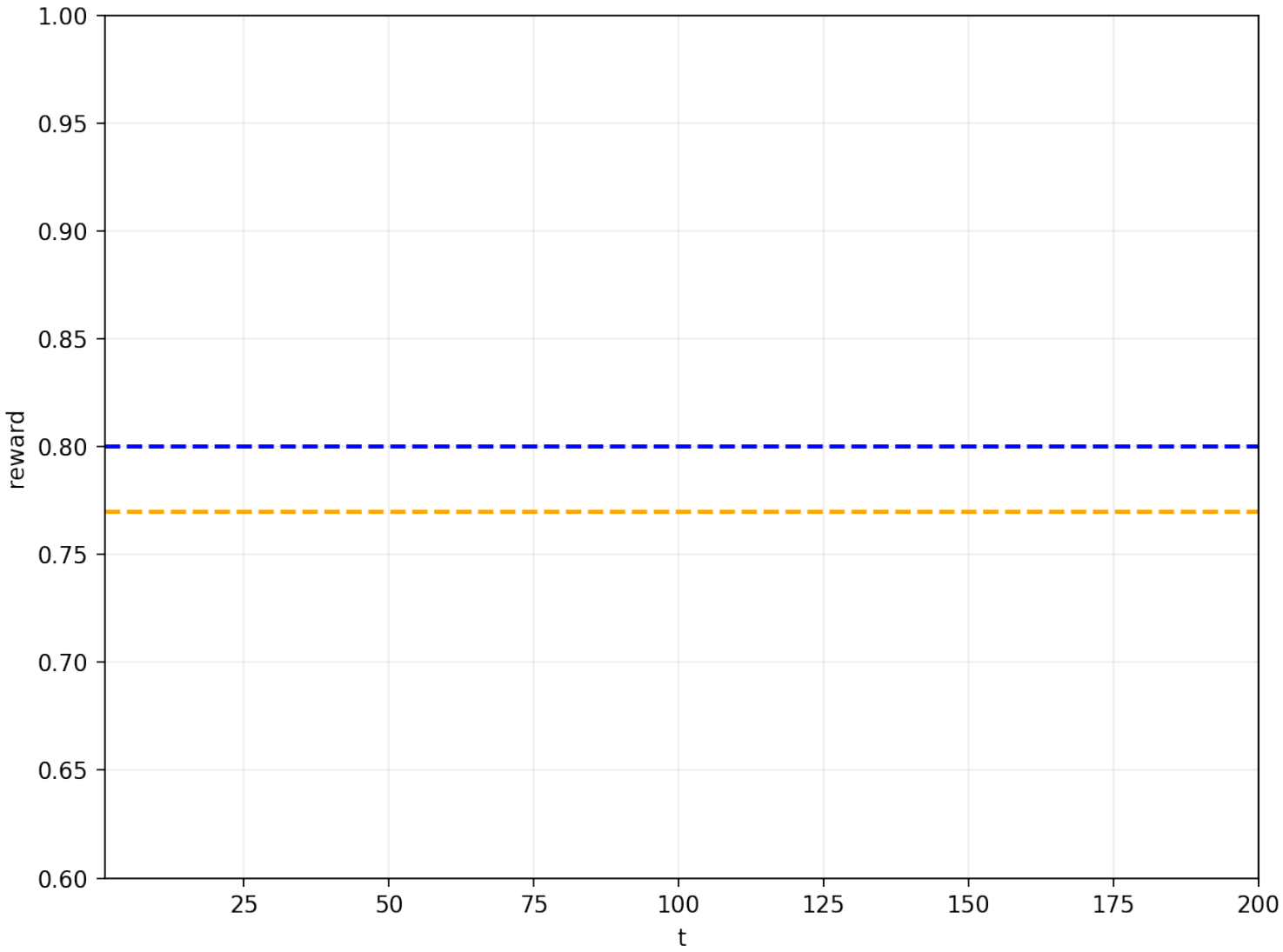
Algorithm

1. **Initialization:** Play each arm once to collect initial rewards.
2. For each round $t = K + 1, \ldots, T$:
   1. Compute $\mathrm{UCB}_a(t)$ for all arms a.
   2. Select the arm $\hat{a}_t = \arg\max_a \mathrm{UCB}_a(t)$.
   3. Observe reward $r_t^{\hat{a}_t}$ and update:

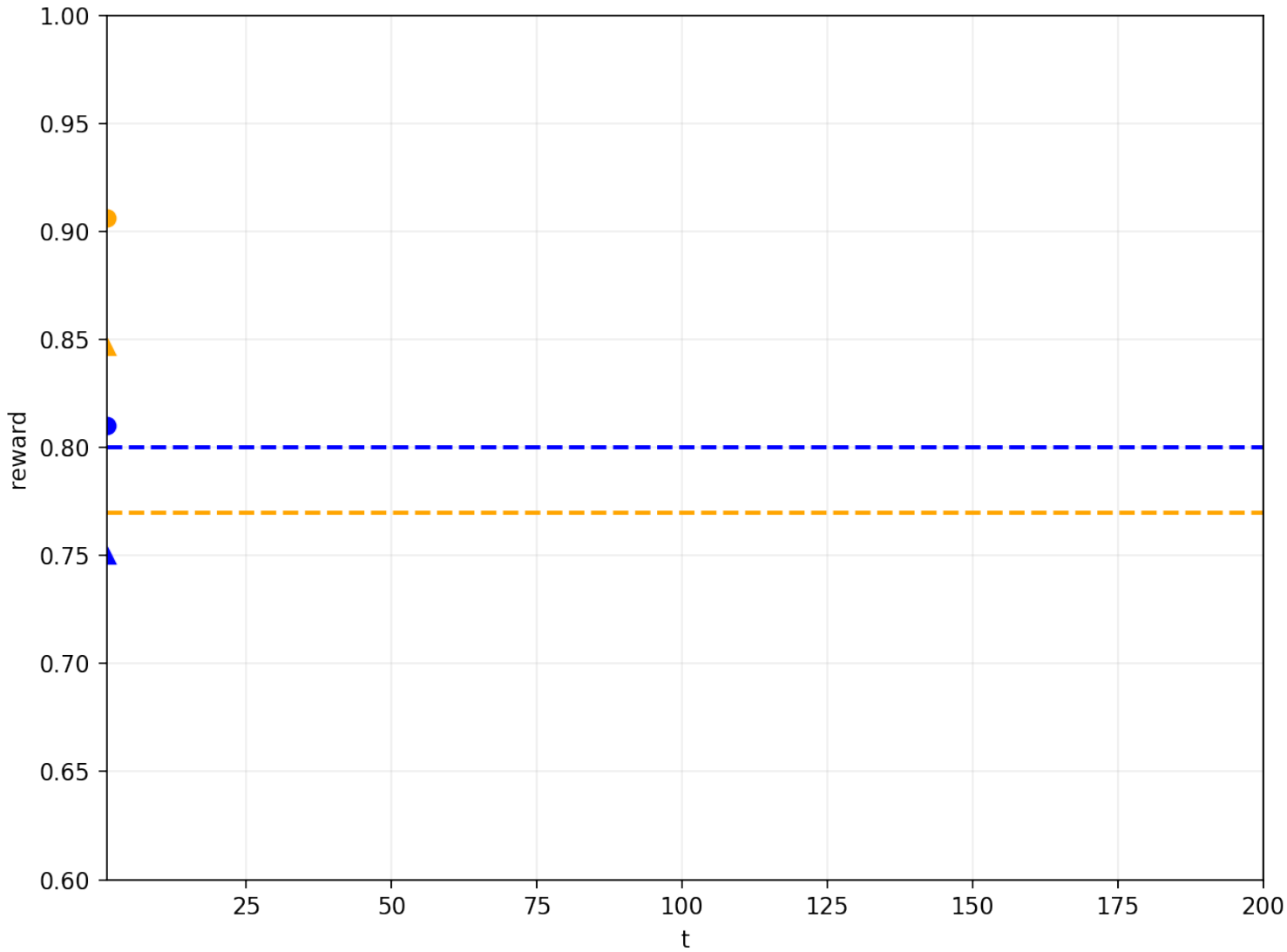
$$\hat{\mu}_{\hat{a}_t}(t+1) = \frac{\hat{\mu}_{\hat{a}_t}(t)\, n_{\hat{a}_t}(t) + r_t^{\hat{a}_t}}{n_{\hat{a}_t}(t) + 1} \quad \text{and} \quad n_{\hat{a}_t}(t+1) = n_{\hat{a}_t}(t) + 1.$$

$$\mathbb{E}[R(T)] = O(\log T).$$

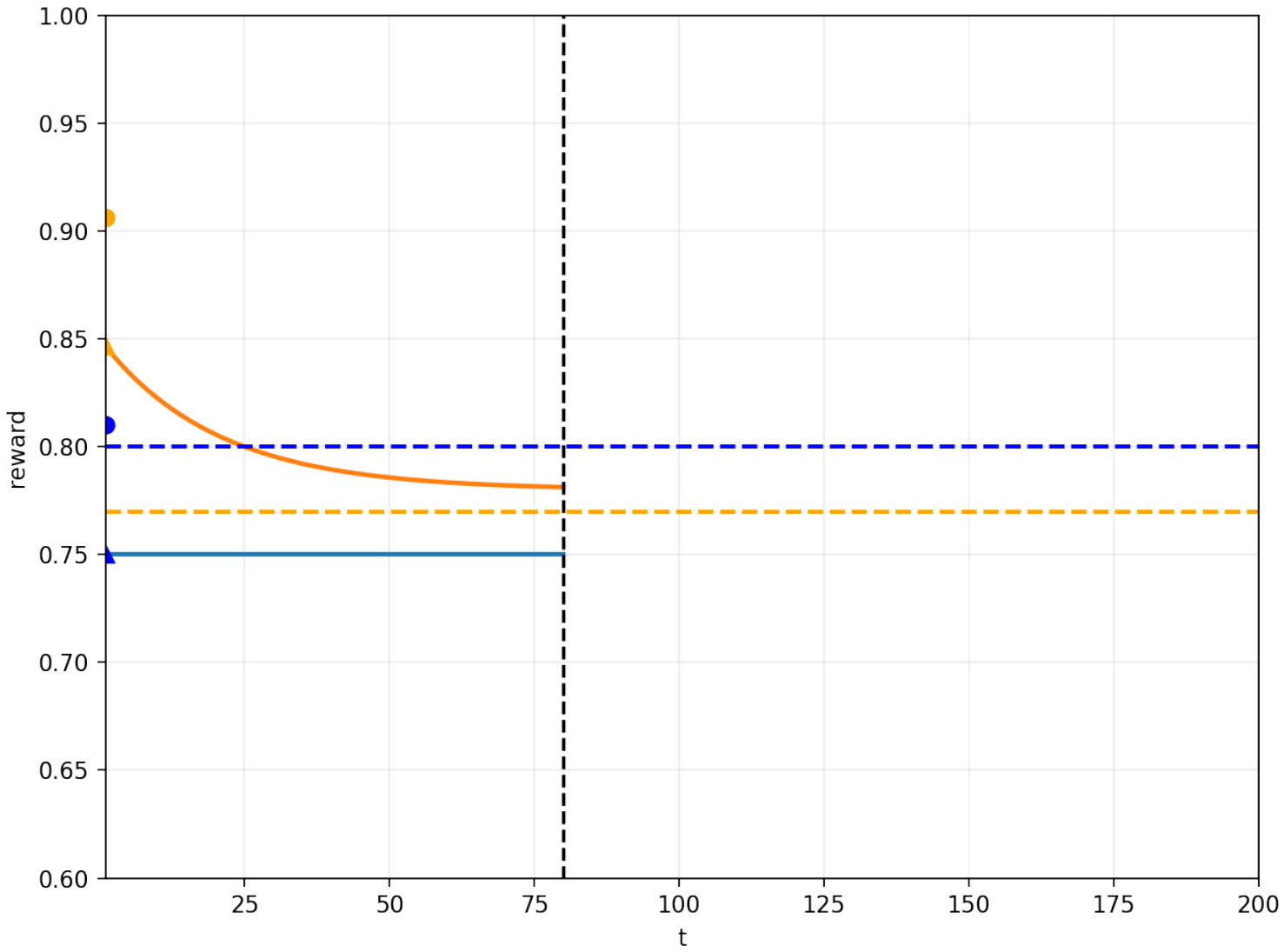
Upper Confidence Bound (UCB1)



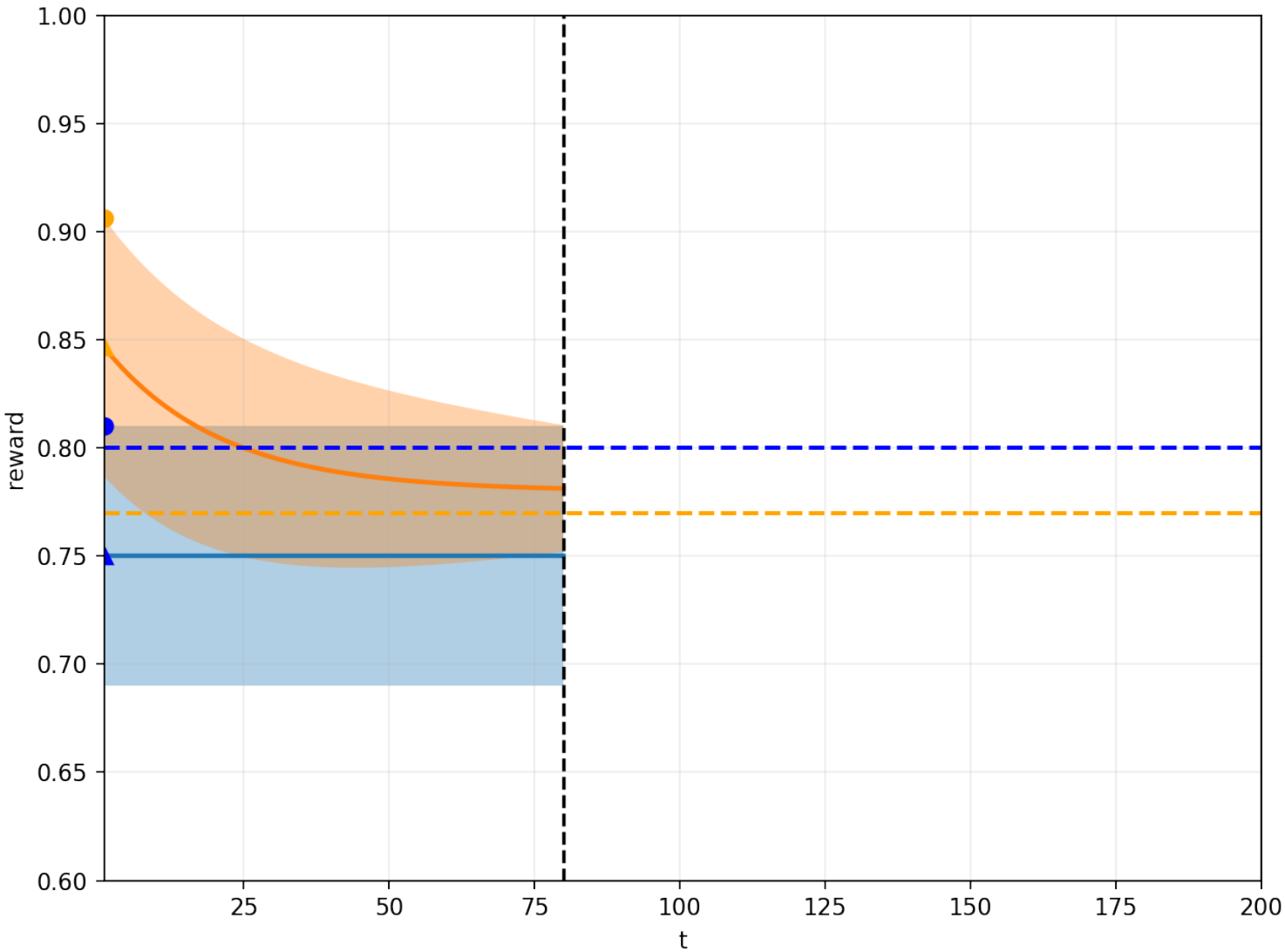
Upper Confidence Bound (UCB1)



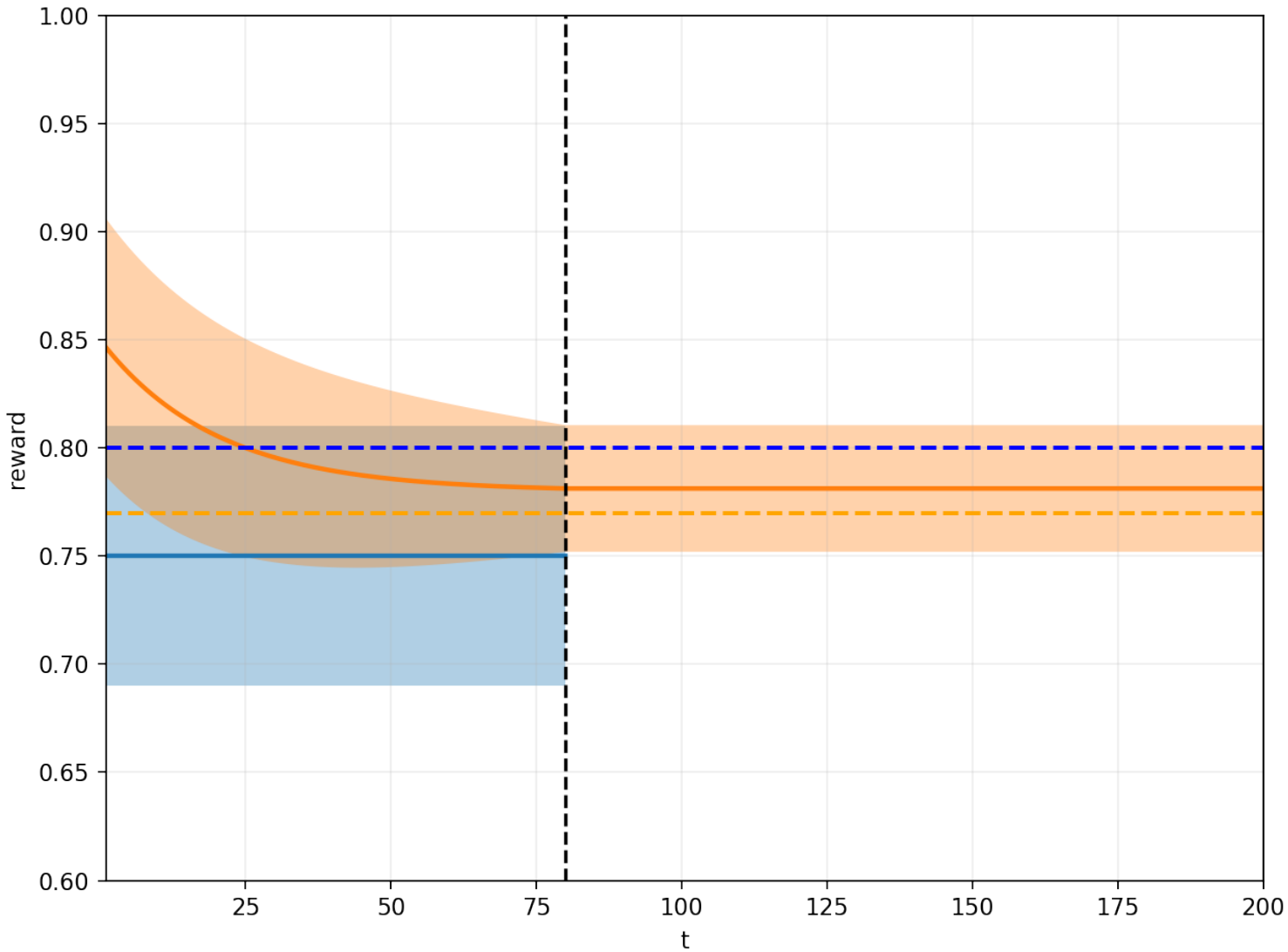
Upper Confidence Bound (UCB1)



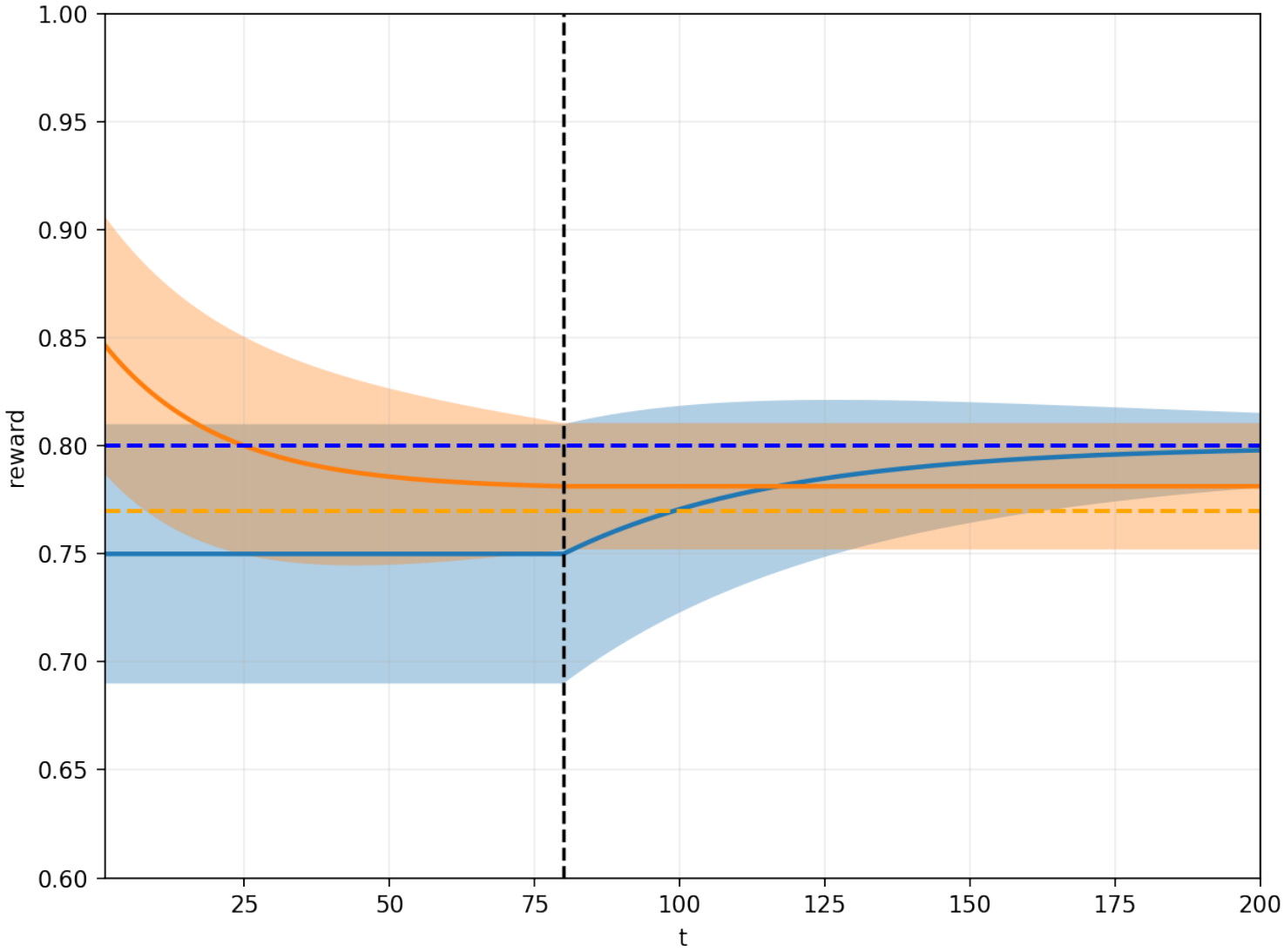
Upper Confidence Bound (UCB1)



Upper Confidence Bound (UCB1)



Upper Confidence Bound (UCB1)



Multi-Armed Bandits in Action

Application Area	Example Bandit Problem
Healthcare	Which drug should a patient receive in a clinical trial?
E-commerce	Which price should we offer to maximize profit?
Online advertising	Which ad should we show to this user right now?
Search	How should we rank the search results for this query?
Recommender systems	Which product or movie should we recommend?
Web design	Which button color or layout gets more clicks?
Crowdsourcing	Which task should go to which worker, and at what price?
Robotics	Which strategy should the robot try in this environment?
Networking	Which settings maximize connection speed or quality?

The magic of Multi-Armed Bandits:

balancing exploration and exploitation

to make smarter, adaptive decisions.

Simple. Powerful. Universal.

References

Auer P, Cesa-Bianchi N, Fischer P (2002) Finite-time analysis of the multiarmed bandit problem. *Machine learning* 47(2-3):235–256.

Cesa-Bianchi N, Fischer P (1998) Finite-time regret bounds for the multiarmed bandit problem. *ICML*, volume 98, 100–108 (Citeseer).

Vermorel J, Mohri M (2005) Multi-armed bandit algorithms and empirical evaluation. *European conference on machine learning*, 437–448 (Springer).